# PROJECT REPORT

## 1. Project Title

Property Maintenance Management

## 2. Student Details

Enrollment Number: IU2541230535
Project Code: CPP60

## 3. Problem Statement

Property owners and managers deal with many linked pieces of information: who owns a property, who is renting it, what its rent is, and which repair complaints are still open. When this information is kept on paper or in scattered files, records get lost or duplicated, a property can be given to two tenants, repair requests are forgotten, and nobody can quickly tell how much has been spent on maintenance.

This project is a console-based C++ application that keeps owners, tenants, properties and maintenance requests in one place, checks every input, links the records to each other, and stores everything in files so that the data is still available the next time the program is opened.

## 4. Objectives

- Store and manage Owner, Tenant, Property and Maintenance Request records in one system.
- Support the complete set of record operations: add, display, search, update and delete.
- Handle the main process of the system: raising a maintenance request for a property and completing it with a final cost.
- Validate all user input (unique IDs, 10-digit phone numbers, e-mail format, positive rent, non-negative cost) and report errors without crashing the program.
- Keep related records consistent (a property cannot have two tenants, an owner who still owns properties cannot be deleted).
- Generate a summary report of rent income, occupied and vacant properties and maintenance cost.
- Save data to text files and load it again at start-up so the data persists.
- Apply object-oriented programming: classes, encapsulation, constructors and destructors, inheritance, runtime polymorphism, dynamic memory, exception handling and modular .h/.cpp files.

## 5. Features

The program starts by loading the saved data automatically and then shows this main menu:

| Menu | Operation | Description |
|---|---|---|

| Menu | Operation | Description |
|---|---|---|
| 1 | Add Record | Adds an Owner, Tenant or Property. Checks that the ID is unique, the phone number has 10 digits, the e-mail contains "@", the owner exists before a property is added, and a property is not already occupied. |
| 2 | Display Records | Shows all Owners, Tenants, Properties or Maintenance Requests. |
| 3 | Search Record | Finds one record by its ID. |
| 4 | Update Record | Changes name, phone and e-mail of an owner or tenant, or address, type and rent of a property. Pressing Enter keeps the old value. |
| 5 | Delete Record | Deletes a record after confirmation. Blocked when other records depend on it (owner with properties, property with maintenance requests). Deleting a tenant makes the property vacant. |
| 6 | Main Transaction | Raise a maintenance request for a property (status Pending) or complete a request by entering the final repair cost (status Completed). |
| 7 | Report | Counts of owners, tenants and properties; occupied and vacant properties; monthly rent income; pending and completed requests; total maintenance cost; list of pending requests. |
| 8 | Save/Load | Saves the data to the files or reloads it from them. |
| 9 | Exit | Saves the updated records and closes the program. |

## 6. Classes and Responsibilities

The system has the six required classes (Property, Owner, Tenant, Maintenance, PropertySystem, FileManager), plus an abstract base class Person and a small input helper.

| Class | Responsibility | Data members and important functions |
|---|---|---|
| Person (abstract base class) | Common data and behaviour of every person in the system. It cannot be instantiated because it has pure virtual functions. | Data: # id, # name, # phone<br>Functions: Person(), Person(id, name, phone), virtual ~Person(), setId(), setName(), setPhone() (with validation), display(), virtual getType() = 0, virtual toFileString() = 0 |
| Owner (derives from Person) | A person who owns one or more properties. | Data: - email<br>Functions: Owner(), Owner(id, name, phone, email), ~Owner(), setEmail(), display(), getType() returns "Owner", toFileString() |
| Tenant (derives from Person) | A person who rents a property. | Data: - propertyId (0 = no property)<br>Functions: Tenant(), Tenant(id, name, phone, propertyId), ~Tenant(), setPropertyId(), display(), getType() returns "Tenant", toFileString() |
| Property | Represents a rentable property and links it to its owner and current tenant. | Data: - id, - address, - propertyType, - ownerId, - tenantId (0 = vacant), - monthlyRent<br>Functions: Property(), Property(id, address, type, ownerId, rent), ~Property(), setters with validation, display(), toFileString() |
| Maintenance | Represents one repair request raised for a property. | Data: - id, - propertyId, - description, - status ("Pending" or "Completed"), - cost<br>Functions: Maintenance(), Maintenance(id, propertyId, description, status, cost), ~Maintenance(), isCompleted(), completeRequest(cost), display(), toFileString() |

| Class | Responsibility | Data members and important functions |
|---|---|---|
| PropertySystem | The controller. Owns all records in dynamic arrays and contains the business logic for every menu option. | Data: - Person** people, - Property* properties, - Maintenance* requests, counts and capacities, - FileManager fileManager<br>Public: addRecord(), displayRecords(), searchRecord(), updateRecord(), deleteRecord(), mainTransaction(), report(), saveData(), loadData()<br>Private helpers: find...Index(), remove...At(), grow...(), clearAll() |
| FileManager | Reads and writes the data files. It only deals with files; it holds no business rules. | Data: - peopleFile, - propertyFile, - requestFile<br>Functions: saveAll(system), loadAll(system) |
| Input (helper)<br>Input.h / Input.cpp | Safe keyboard input functions used by the menus. | readLine(prompt), readInt(prompt), readDouble(prompt) - keep asking until the value is valid |
| main.cpp | Entry point. Creates the PropertySystem object, shows the menu and calls the system functions. | main(), showMenu() |

## 7. OOP Concepts Used

### 7.1 Classes and Objects

Each real-world thing is a class and every record is an object. For example, a property record is created as an object and stored in the system:

```
Property prop(101, "Flat 12, Shivalik Apartments", "Flat", 1, 15000);
storeProperty(prop);
```

### 7.2 Encapsulation

Data members are private or protected. They can only be changed through setter functions, which validate the value before accepting it:

```
void Property::setMonthlyRent(double monthlyRent) {
    if (monthlyRent <= 0) {
        throw invalid_argument("Monthly rent must be greater than 0.");
    }
    this->monthlyRent = monthlyRent;
}
```

### 7.3 Constructors and Destructors

Every class has a default constructor and a parameterized constructor. The parameterized constructors call the validating setters. Destructors are defined for every class; the Person destructor is virtual so that Owner and Tenant objects are destroyed correctly through a Person pointer. PropertySystem has a destructor that releases all dynamic memory.

```
Owner::Owner() : Person(), email("") { }
Owner::Owner(int id, const string& name, const string& phone, const string& email)
    : Person(id, name, phone), email("") { setEmail(email); }
```

### 7.4 Inheritance

Owner and Tenant inherit the common members (id, name, phone) from the base class Person, so this code is written only once:

```
class Owner  : public Person { ... };
class Tenant : public Person { ... };
```

### 7.5 Runtime Polymorphism

Person declares virtual functions (display, getType, toFileString). The people array holds Person pointers that point to Owner or Tenant objects, and the correct version of the function is chosen at run time. The same call is used when displaying and when saving to file:

```
for (int i = 0; i < peopleCount; i++) {
    people[i]->display();              // Owner::display() or Tenant::display()
}
pf << system.getPerson(i)->toFileString() << "\n";   // Owner or Tenant line
```

### 7.6 Dynamic Memory

Records are kept in dynamic arrays created with new, so the system is not limited to a fixed number of records. When an array becomes full, a bigger array of double the size is created, the old elements are copied and the old array is released with delete[]. Owner and Tenant objects are created with new and deleted when removed. All memory is released in the destructor.

```
people = new Person*[peopleCapacity];          // PropertySystem constructor
properties = new Property[propertyCapacity];
requests = new Maintenance[requestCapacity];

Person** bigger = new Person*[newCapacity];    // growPeople(): array was full
for (int i = 0; i < peopleCount; i++) bigger[i] = people[i];
delete[] people;
people = bigger;

PropertySystem::~PropertySystem() {            // destructor
    clearAll();          // delete every Owner / Tenant object
    delete[] people;
    delete[] properties;
    delete[] requests;
}
```

Purpose: memory is used only as needed, the number of records can grow while the program is running, and memory is returned to the system when the program ends.

### 7.7 Validation and Exception Handling

Setters throw exceptions (invalid_argument, logic_error, runtime_error) for invalid data. Each menu function catches them with try/catch and shows a clear message, so wrong input never crashes the program:

```
try {
    if (choice == 1) addOwnerInteractive();
    ...
} catch (const exception& e) {
    cout << "Error: " << e.what() << "\n";
}
```

### 7.8 Modular Programming

Every class is split into a header file (.h) and an implementation file (.cpp). main.cpp only creates the PropertySystem object, shows the menu and calls the system functions; the business logic is inside the classes.

## 8. File Handling

The system stores three text files in the Data folder. Each record is one line and the fields are separated by the "|" character. Because "|" is the separator, the setters reject any text that contains it.

| File | Content | Line format |
|---|---|---|
| Data/people.txt | Owners and Tenants (O or T at the start of the line tells the loader which object to create) | O\|id\|name\|phone\|email<br>T\|id\|name\|phone\|propertyId |
| Data/properties.txt | Properties | id\|address\|type\|ownerId\|tenantId\|monthlyRent |
| Data/requests.txt | Maintenance requests | id\|propertyId\|description\|status\|cost |

Example lines:

```
O|1|Rahul Mehta|9876543210|rahul.mehta@gmail.com
T|4|Priya Shah|9123456780|101
101|Flat 12, Shivalik Apartments, Satellite|Flat|1|4|15000.000000
1|101|Water leakage in bathroom|Completed|2500.000000
```

### Saving

When the user chooses Save (menu 8) or Exit (menu 9), PropertySystem::saveData() calls FileManager::saveAll(). For each person the function calls the virtual toFileString(), so Owner and Tenant write their own line format. If a file cannot be opened (for example the Data folder is missing), a runtime_error is thrown and a message is shown.

### Loading

At start-up the program calls loadData(), which calls FileManager::loadAll(). Each line is split at "|" and the matching object is created with new (an Owner for "O", a Tenant for "T"). Lines that are damaged or fail validation are skipped with a message instead of stopping the program. If the files do not exist (first run), the program starts with no records.

### Persistence testing

1. Add some records (owner, property, tenant, maintenance request).

2. Choose 8 then 1 (Save), or choose 9 (Exit). The message "Data saved successfully." appears and the three files exist in the Data folder.
3. Close the program and run it again.
4. The start-up message shows the loaded record counts ("Data loaded: X people, Y properties, Z requests.") and Display Records shows the same data as before.

## 9. UML Diagrams

### 9.1 Use Case Diagram

The Property Manager operates the system and uses all functions. The Tenant raises maintenance requests and the Property Owner views records and reports. Input validation is included by the add, update and request use cases.

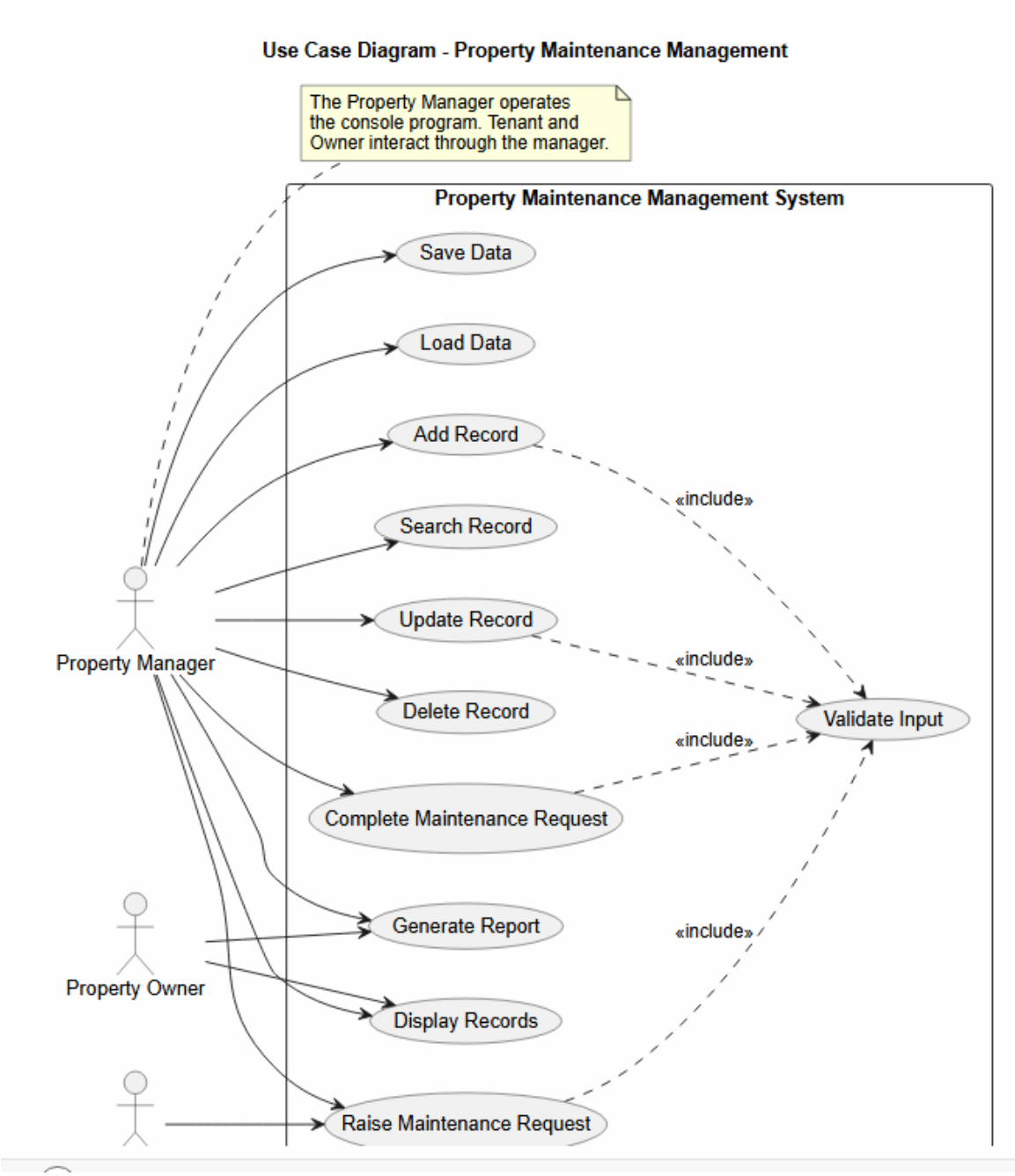



Figure 9.1: Use Case Diagram

## 9.2 Class Diagram

The diagram shows the classes with their data members, member functions, visibility (+ public, - private, # protected) and relationships: Owner and Tenant inherit from Person, PropertySystem aggregates people, properties and maintenance requests and is composed with a FileManager, and Property is linked to its Owner and Tenant.

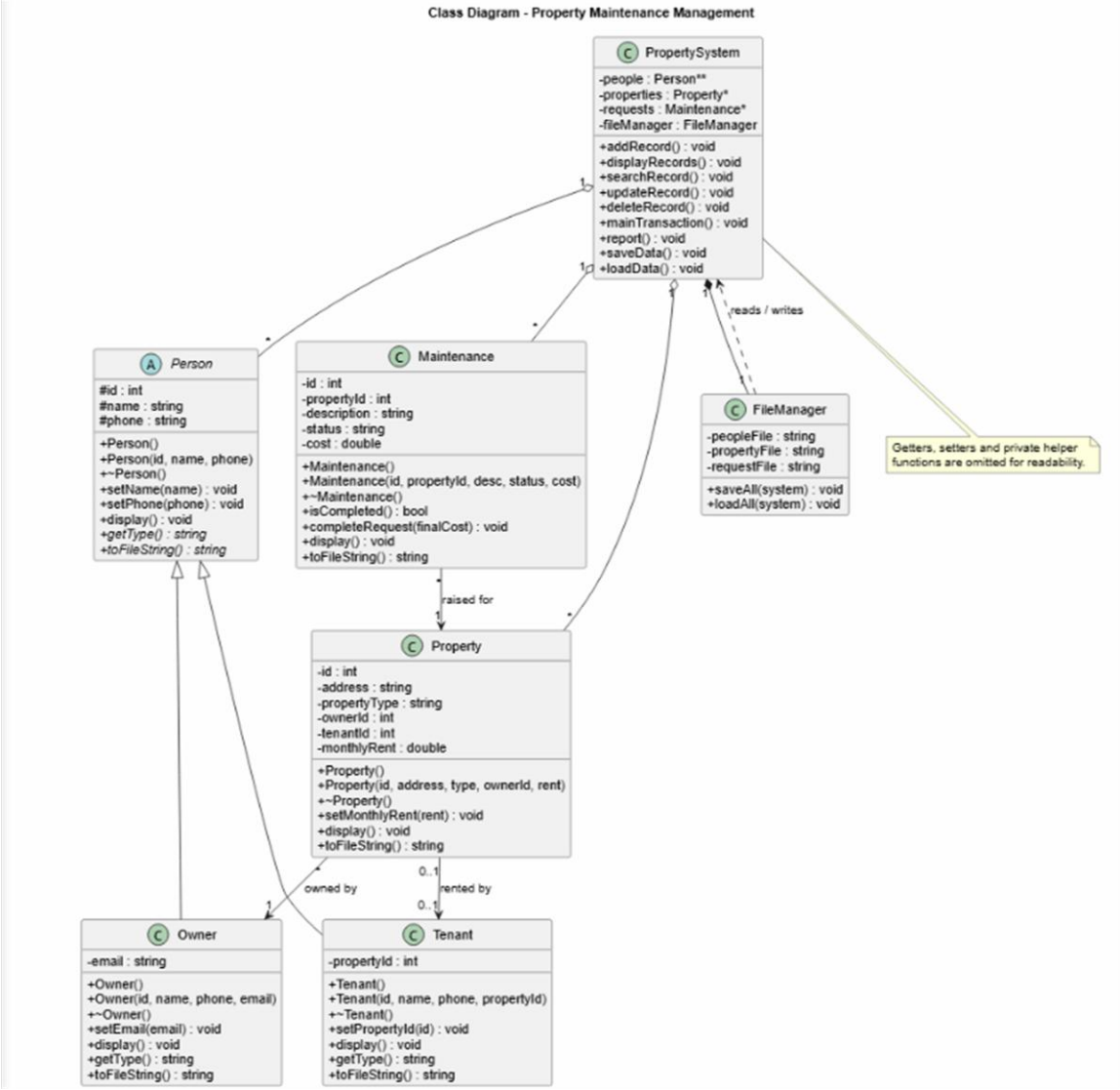



Figure 9.2: Class Diagram

## 9.3 Sequence Diagram 1 - Main Process: Raise and Complete Maintenance Request

Raising a request: the manager chooses menu 6, PropertySystem reads the request ID and property ID, checks that the ID is new and the property exists, creates the Maintenance object with status Pending and stores it (growing the array if full).

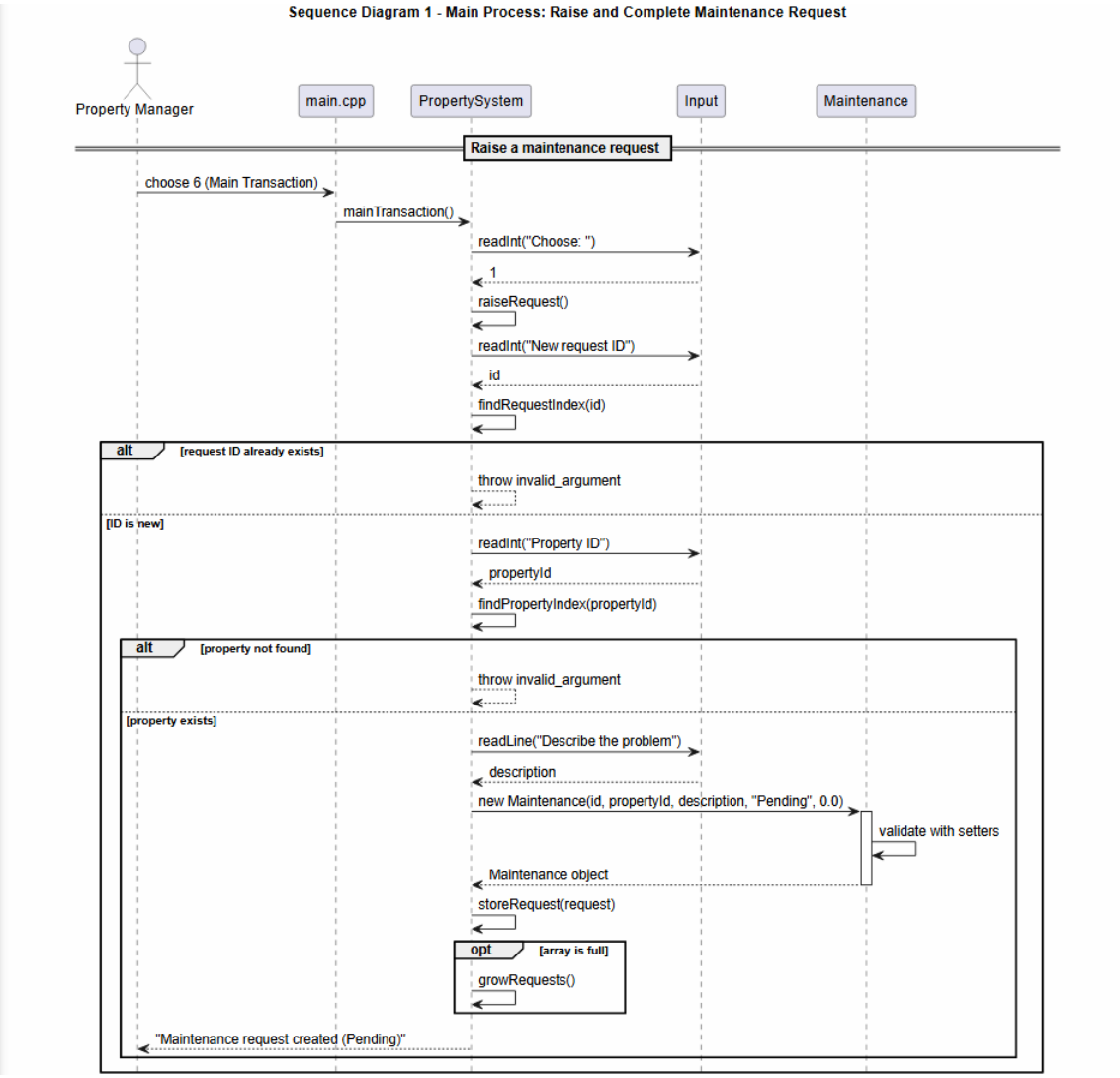



**Figure 9.3a: Sequence Diagram 1 (part 1) - Raise a maintenance request**

Completing a request: PropertySystem finds the request, reads the final cost and calls Maintenance::completeRequest(), which rejects an already completed request and otherwise sets the cost and the status to Completed.

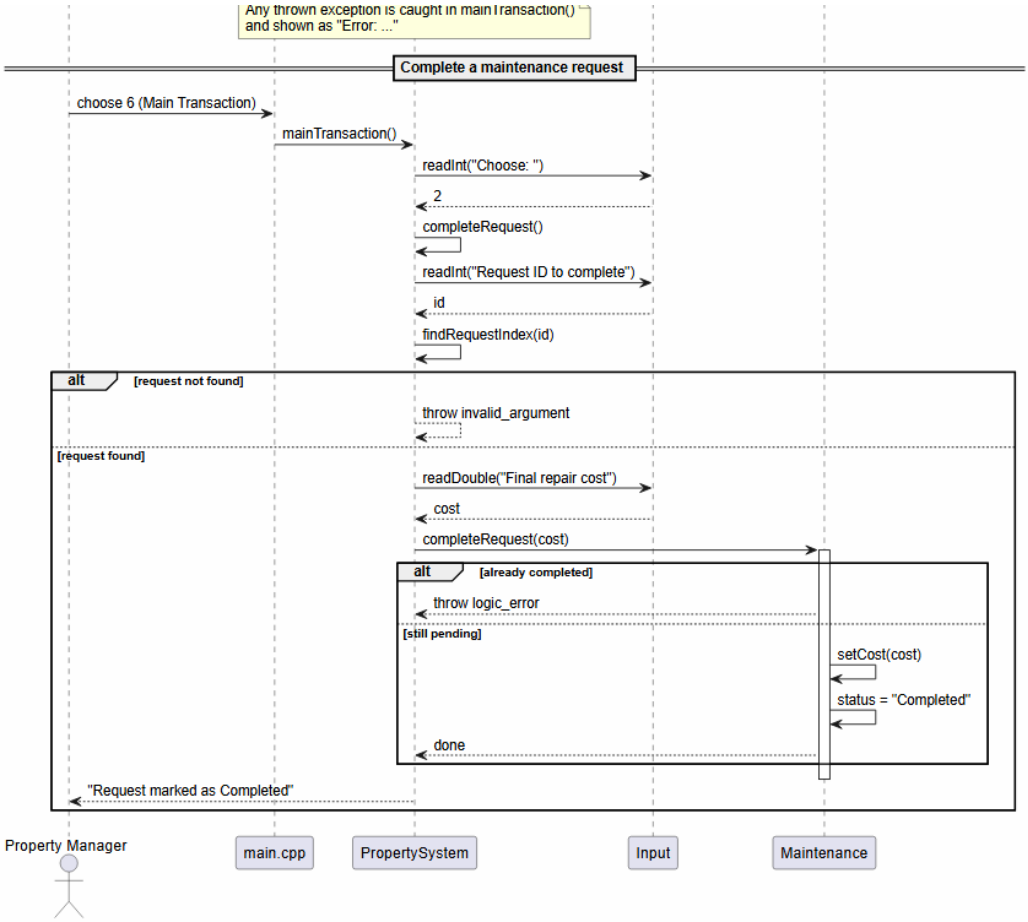



Figure 9.3b: Sequence Diagram 1 (part 2) - Complete a maintenance request

## 9.4 Sequence Diagram 2 - Save Data to Files

PropertySystem calls FileManager::saveAll(). For each person, the virtual toFileString() is called (runtime polymorphism), and properties and requests are written to their own files. A runtime_error is thrown if a file cannot be opened.

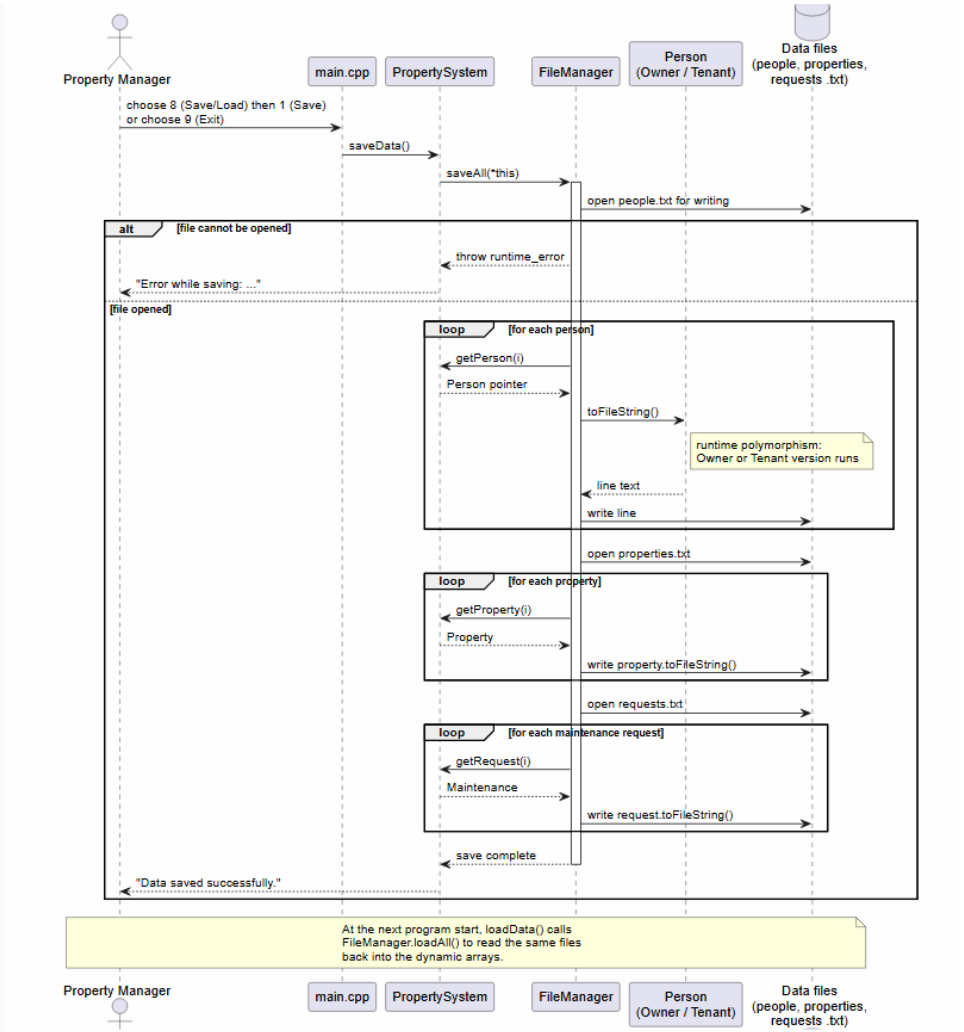



**Figure 9.4: Sequence Diagram 2 - Save data (persistence)**

## 10. Program Workflow

*USER > MAIN MENU > PROPERTYSYSTEM > ENTITY CLASSES > MAIN PROCESS > FILEMANAGER > DATA FILES*

1. The program starts and main() creates the PropertySystem object. Its constructor allocates the dynamic arrays.
2. main() calls loadData(). FileManager reads Data/people.txt, Data/properties.txt and Data/requests.txt and creates the objects. The number of loaded records is shown.
3. The main menu (1-9) is displayed in a loop. Input is read safely; a wrong choice shows a message and the menu is shown again.
4. The chosen option calls the matching PropertySystem function: add, display, search, update, delete, main transaction, report or save/load.
5. PropertySystem works with the entity objects (Owner, Tenant, Property, Maintenance). Validation errors are thrown as exceptions and shown as "Error: ..." messages.

6. In the main transaction a maintenance request is raised for an existing property (Pending) and later completed with a final cost (Completed).
7. Save/Load (menu 8) calls FileManager to write or reload the data files.
8. Choosing Exit (menu 9) saves all records through FileManager and ends the loop. The PropertySystem destructor deletes all objects and releases the dynamic arrays.
9. On the next run, the saved records are loaded again.

## 11. Screenshots

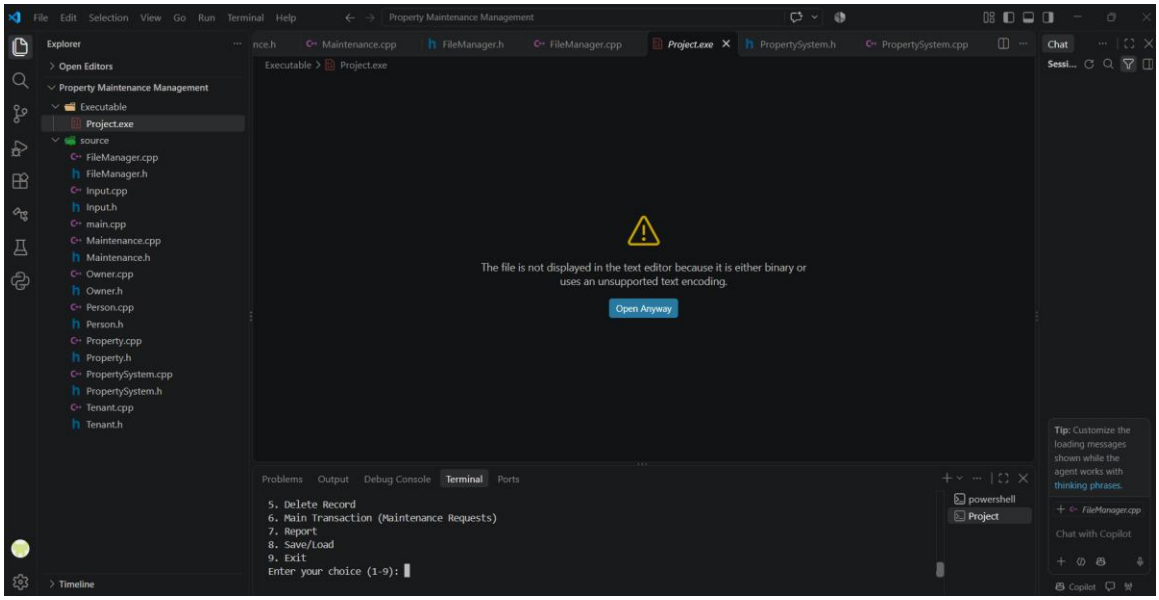



Figure 11.1: Project files in VS Code and the program running with the main menu

```
8. Save/Load
9. Exit
Enter your choice (1-9): 6

--- Maintenance Process ---
1. Raise a new maintenance request
2. Complete a maintenance request
Choose: 2
Request ID to complete: 2365435425
Please enter a valid whole number.
Request ID to complete: 354535356
Error: Request not found.

==================================
   PROPERTY MAINTENANCE MANAGEMENT
==================================
1. Add Record
2. Display Records
3. Search Record
```

Figure 11.2: Input validation and exception handling in the main transaction (invalid number rejected, unknown request ID reported)

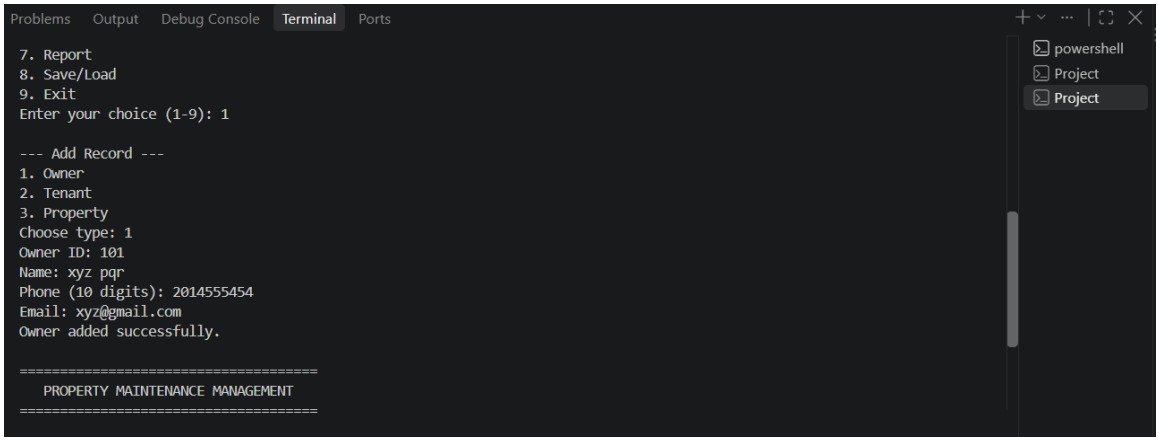



Figure 11.3: Adding records

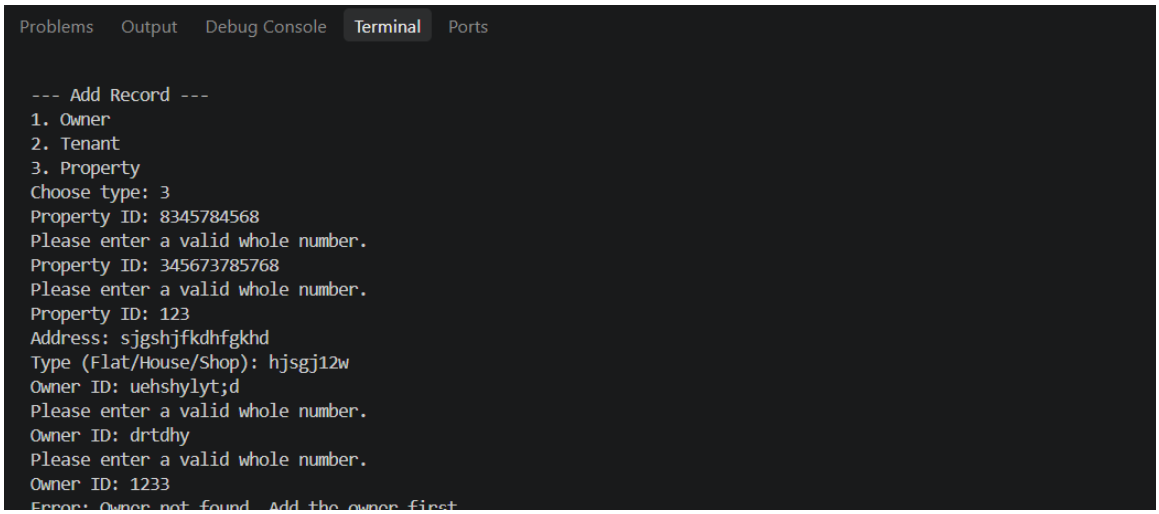



Figure 11.4: Display and search

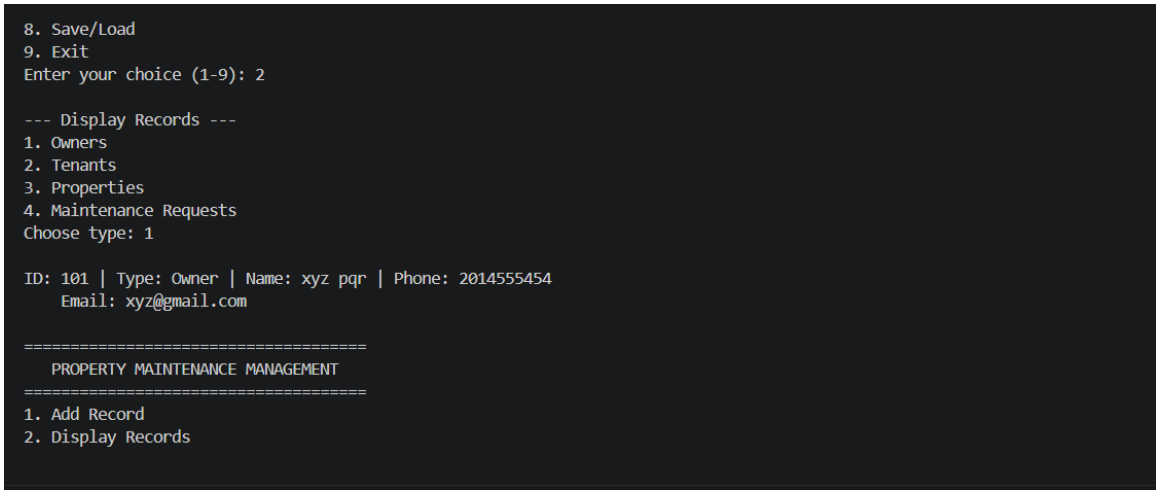



Figure 11.5: Main transaction

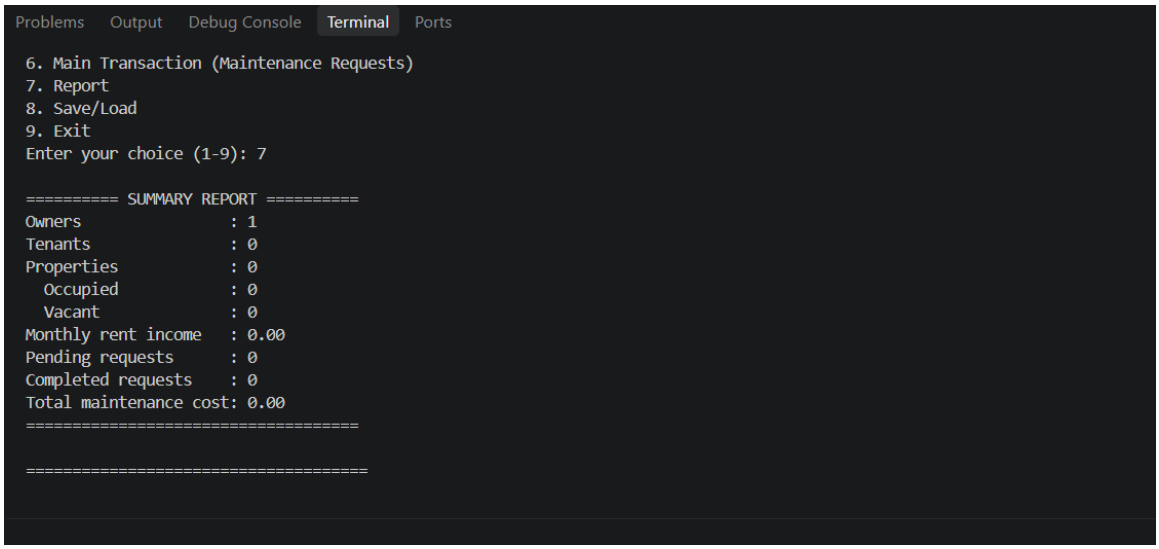



Figure 11.6: Summary report

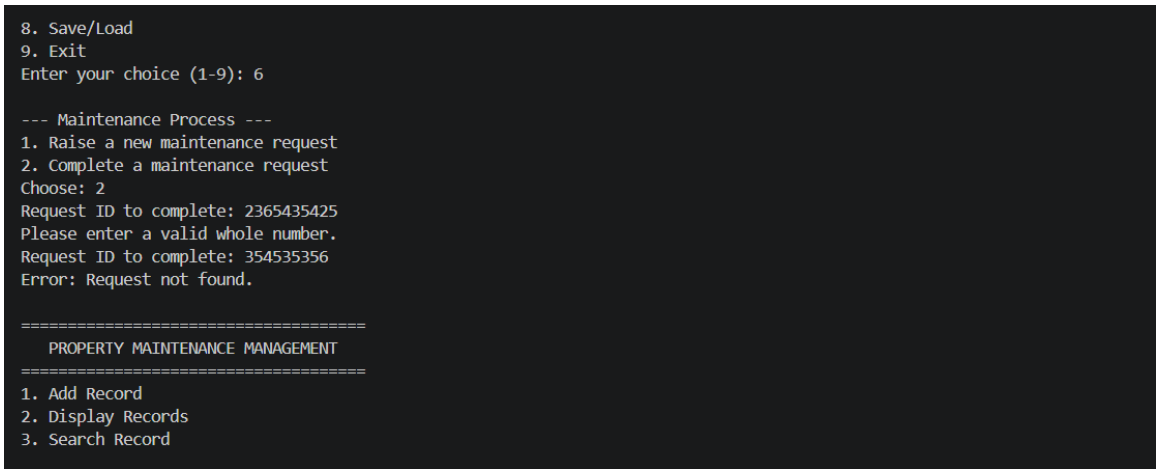



Figure 11.7: File persistence

## 12. Testing

Expected output is what the program is designed to print. Fill the Actual output column from your own run (highlighted cells) and mark Pass or Fail.

| No. | Test case | Input | Expected output | Actual output |
|---|---|---|---|---|
| 1 | Start with no data files | Run Project.exe on the first run | Data loaded: 8 people, 6 properties, 6 requests. | Data loaded: 8 people, 6 properties, 6 requests. |
| 2 | Add a valid owner | 1, 1, ID 1, Rahul Mehta, 9876543210, rahul@gmail.com | Owner added successfully. | Owner added successfully. |
| 3 | Duplicate person ID | 1, 1, ID 1 | Error: That ID is already used by another person. | Error: That ID is already used by another person. |
| 4 | Invalid phone number | 1, 1, ID 5, name, phone 12345 | Error: Phone number must be exactly 10 digits. | Error: Phone number must be exactly 10 digits. |
| 5 | Property with a missing owner | 1, 3, ID 102, address, Flat, owner ID 99 | Error: Owner not found. Add the owner first. | Error: Owner not found. Add the owner first. |
| 6 | Tenant for an occupied property | Add tenant with property ID of an occupied property | Error: That property already has a tenant. | Error: That property already has a tenant. |

| No. | Test case | Input | Expected output | Actual output |
|---|---|---|---|---|
| 7 | Raise a maintenance request | 6, 1, request ID 1, property ID 101, "Water leakage" | Maintenance request created (status: Pending). | Maintenance request created (status: Pending). |
| 8 | Complete the request | 6, 2, request ID 1, cost 2500 | Request marked as Completed. | Request marked as Completed. |
| 9 | Complete it again | 6, 2, request ID 1, cost 100 | Error: This request is already completed. | Error: This request is already completed. |
| 10 | Raise request for a missing property | 6, 1, request ID 2133212, property ID 32344323 | Error: Property not found. | Error: Property not found. Pass |
| 11 | Number too large for an int | 6, 2, request ID 2365435425 | Please enter a valid whole number. (asks again) | Please enter a valid whole number.<br>Prompt repeated. Pass |
| 12 | Complete a request that does not exist | 6, 2, request ID 354535356 | Error: Request not found. | Error: Request not found. Pass |
| 13 | Delete owner who still owns a property | 5, 1, owner ID 1 | Error: This owner still owns properties. Delete those first. | Error: This owner still owns properties. Delete those first. |
| 14 | Report | 7 | Counts of owners, tenants, properties, occupied/vacant, rent income, pending/completed requests and total cost | ========== SUMMARY REPORT ==========<br>Owners : 3<br>Tenants : 5<br>Properties : 6<br>Occupied : 4<br>Vacant : 2<br>Monthly rent income : 90000.00<br>Pending requests : 3<br>Completed requests : 3<br>Total maintenance cost: 7800.00<br><br>--- Pending Requests ---<br>Request ID: 3 \| Property ID: 103 \| Status: Pending \| Cost: 0.00<br>Problem: Electrical wiring short circuit in hall<br>Request ID: 4 \| Property ID: 104 \| Status: Pending \| Cost: 0.00<br>Problem: Wall painting required before new tenant<br>Request ID: 6 \| Property ID: 106 \| Status: Pending \| Cost: 0.00<br>Problem: Kitchen tap replacement<br>========== |
| 15 | Persistence | Save (8, 1), exit (9), run again | Data loaded: X people, Y properties, Z requests. (same counts as saved) and records are displayed | Data loaded: X people, Y properties, Z requests. (same counts as saved) and records are displayed |

## 13. Challenges and Solutions

| Challenge | Solution |
|---|---|

| Challenge | Solution |
|---|---|
| Compiling failed with "cannot open output file Executable/Project.exe" because the Executable folder did not exist. | The Executable folder was created with mkdir and the compile command was run again. |
| Windows blocked the new Project.exe with an Application Control policy message. | [write here how you solved it] |
| Mixing cin >> and getline leaves a newline in the buffer, and wrong input such as letters breaks the program. | All input goes through the helper functions readLine(), readInt() and readDouble() in Input.cpp, which use getline and stoi/stod inside try/catch and ask again until the value is valid. For example the oversized number 2365435425 is rejected and the user is asked again. |
| The "\|" separator in the data files could be broken by text typed by the user. | The setters reject names, addresses, e-mails and descriptions that contain "\|". |
| PropertySystem needs FileManager, and FileManager needs PropertySystem (circular dependency). | FileManager.h uses a forward declaration of PropertySystem; the full header is included only in FileManager.cpp. |
| Keeping records consistent when something is deleted. | An owner with properties and a property with maintenance requests cannot be deleted. Deleting a tenant or property updates the linked tenantId / propertyId. |
| The number of records is not known in advance. | Dynamic arrays start with a small capacity and double in size when full, and everything is released in the destructor with delete[]. |
| The program must not crash if the data files are missing or damaged. | A missing file is treated as "no data yet"; damaged lines are skipped with a message; failure to write a file throws a runtime_error that is caught and reported. |

## 14. Conclusion

The Property Maintenance Management system manages owners, tenants, properties and maintenance requests through a simple menu, validates all input, and keeps the data in text files so it persists between runs. It also meets the required structure: separate classes in .h/.cpp files, a main.cpp that only creates the system and calls its functions, and a clear flow from the user to the entity classes and the data files.

While developing it I practised classes and objects, encapsulation with validating setters, default and parameterized constructors, destructors, inheritance (Person to Owner and Tenant), runtime polymorphism through virtual functions and Person pointers, dynamic memory with new and delete[], exception handling, and file handling. I also learned how to separate responsibilities between the controller class (PropertySystem), the entity classes and the file class, and how to test a program with both valid and invalid input.

Possible future improvements are searching by name, assigning repair priority and dates to requests, and a graphical or database-backed version.